

ARCHITECTURE HANDBOOK

Perionyx Enterprise Financial Operating System

Master Edition

v1.0.0

2026-07-16 | 56 Business Modules | 272+ API Endpoints
100+ Prisma Models | 443+ Tests

Confidential — Perionyx Platform Core v1.0.0

Table of Contents

1. Table of Contents

2. 1. Executive Overview

- 1.1 What Perionyx Is
- 1.2 Product Philosophy
- 1.3 Platform at a Glance
- 1.4 Core Architectural Guarantees

3. 2. System Architecture

- 2.1 Seven-Layer Architecture
- 2.2 Proxy Architecture
- 2.3 Key Architectural Decisions

4. 3. Technology Stack

- 3.1 Core Framework
- 3.2 Infrastructure Components
- 3.3 Why These Technologies

5. 4. Domain Architecture & Business Modules

- 4.1 Domain Map
- 4.2 Module Interaction Pattern

6. 5. Financial Engine — Ledger & Accounting

- 5.1 Double-Entry Accounting
- 5.2 Journal Lifecycle
- 5.3 Six-Stage Processing Pipeline
- 5.4 Invariant

7. 6. Treasury Platform

- 6.1 Core Models
- 6.2 Cash Management
- 6.3 Liquidity Tiers
- 6.4 Bank Integrations
- 6.5 Cross-Domain Interactions

8. 7. Reporting & Analytics Platform

- 7.1 Report Engine Pipeline
- 7.2 Supported Reports
- 7.3 13 Analytics Components
- 7.4 Rendering Strategy

9. 8. Integration Platform

- 8.1 Architecture

8.2 Connector Framework

8.3 Built-in Connectors

8.4 Data Pipeline

8.5 Lineage & Audit

10. 9. Intelligence Platform

9.1 Architecture

9.2 Six Deterministic Engines

9.3 Engine Pattern

9.4 KPI Framework

9.5 Scorecard Service

9.6 Explain Engine

11. 10. Workflow Orchestration

10.1 Components

10.2 Step Types

10.3 Workflow Templates (10)

10.4 Trigger Types

12. 11. Enterprise Experience Platform

11.1 Role Dashboards

11.2 Enterprise Form System (14 components)

11.3 Enterprise Motion (13 components)

11.4 Mobile Experience (9 components)

11.5 Onboarding Module

13. 12. Security Architecture

12.1 Authentication

12.2 Authorization — RBAC + ABAC

12.3 Endpoint Protection Pattern

12.4 Encryption

12.5 Audit Logging

12.6 Security Headers

14. 13. Multi-Tenancy Architecture

13.1 Tenant Model

13.2 Tenant Context Propagation

13.3 Cross-Tenant Blocking (Four Layers)

13.4 Scalability

15. 14. Database Architecture

14.1 Schema Overview

14.2 Schema Design Principles

14.3 Index Strategy

14.4 JSONB Usage

16. 15. Queue & Async Processing

15.1 Two-Layer Queue Architecture

- 15.2 Job Types (11)
- 15.3 Default Infrastructure Queues
- 15.4 Key Features

17. 16. API Architecture

- 16.1 Handler Pattern
- 16.2 Request Flow
- 16.3 Unified Error Handling
- 16.4 Cache Strategy
- 16.5 API Endpoint Count

18. 17. Deployment Architecture

- 17.1 Docker Multi-Stage Build
- 17.2 Health Endpoints
- 17.3 Graceful Shutdown
- 17.4 Recovery Module
- 17.5 CI/CD Pipeline
- 17.6 Monitoring

19. 18. Scalability & Performance Strategy

- 18.1 Caching Architecture
- 18.2 TTL Strategy
- 18.3 Database Optimization
- 18.4 Lazy Loading Strategy
- 18.5 Future Scalability

20. 19. AI Governance & Intelligence

- 19.1 Non-Negotiable Principle
- 19.2 What AI May Do
- 19.3 What AI May Never Do
- 19.4 Evidence Requirements
- 19.5 Confidence Levels
- 19.6 Enterprise Trust Model

21. 20. Engineering Standards & Constitutions

- 20.1 Perionyx Engineering Constitution
- 20.2 Performance Constitution
- 20.3 AI Engineering Playbook
- 20.4 Self-Review Framework
- 20.5 Enterprise Readiness Checklist

22. 21. Platform Roadmap

- 21.1 Completed Phases
- 21.2 Future Phases

23. 22. Documentation Index & Cross-References

- 22.1 Architecture Documents
- 22.2 Supporting Documents

22.3 Design Documents

22.4 Domain Documentation

24. 23. Architecture Decision Records (ADRs)

25. 24. Architecture Diagrams

24.1 Seven-Layer Platform Architecture

24.2 Financial Transaction Flow

24.3 Integration Data Flow

24.4 Workflow Engine

24.5 Intelligence Platform

24.6 AI Decision Flow

24.7 Deployment Topology

26. Appendix A: Key File Locations

27. Appendix B: Version & Architecture Freeze

Revision History

Version	Date	Author	Description
1.0.0	2026-07-16	Perionyx Engineering	Master Edition — complete architecture documentation integrating all 23 architecture documents, 30 ADRs, and supporting domain documentation
—	—	—	—

Document Classification: Confidential — Perionyx Internal & Authorized Partners

Document ID: PER-ARCH-HB-1-0-0

Status: Ratified

Version: 1.0.0 **Status:** Ratified **Audience:** CTO, CFO, Engineering, Architecture Review, DevOps, Compliance **Last Updated:** 2026-07-16

Table of Contents

- 1. Executive Overview
- 2. System Architecture
- 3. Technology Stack
- 4. Domain Architecture & Business Modules
- 5. Financial Engine — Ledger & Accounting
- 6. Treasury Platform
- 7. Reporting & Analytics Platform
- 8. Integration Platform
- 9. Intelligence Platform
- 10. Workflow Orchestration
- 11. Enterprise Experience Platform
- 12. Security Architecture
- 13. Multi-Tenancy Architecture
- 14. Database Architecture
- 15. Queue & Async Processing
- 16. API Architecture
- 17. Deployment Architecture
- 18. Scalability & Performance Strategy
- 19. AI Governance & Intelligence
- 20. Engineering Standards & Constitutions
- 21. Platform Roadmap
- 22. Documentation Index & Cross-References
- 23. Architecture Decision Records (ADRs)
- 24. Architecture Diagrams

1. Executive Overview

1.1 What Perionyx Is

Perionyx is an enterprise financial operations platform purpose-built for CFOs, treasurers, controllers, finance managers, and auditors. It unifies treasury management, double-entry ledger, financial reporting, risk management, compliance monitoring, intelligence, and workflow automation into a single multi-tenant platform.

Positioning statement: *Bloomberg Terminal meets Stripe Dashboard for enterprise finance.*

1.2 Product Philosophy

Five ordered principles drive every architectural and design decision:

Principle	Definition
Clarity	Every screen answers one question; no visual noise. Metrics render first, explanations second.
Confidence	Stale data is labeled. Destructive actions require confirmation. Cached balances are marked.
Speed	CFOs don't wait. Metric values render before charts. Page transitions complete in under 400ms.
Beauty	Achieved through restraint — generous whitespace, consistent rhythm, purposeful color.
Trust	Every number has a source. Every state has an explanation. Audit trails are unbroken.

1.3 Platform at a Glance

Metric	Value
Platform Version	v1.0.0
Release Date	2026-07-09
Business Modules	56 (<code>src/modules/</code>)
API Endpoints	272+
Pages	96+
Prisma Models	100+
Permissions	46+ <code>GranularPermission</code> types
Intelligence Engines	6 deterministic engines
Test Count	443+ (all passing)
Zero TypeScript Errors	Enforced at build

Source: `docs/architecture/01-executive-overview.md`

1.4 Core Architectural Guarantees

- Multi-tenant by design** — every table carries `companyId`; `requireTenantContext()` enforces isolation.
- Financial facts are deterministic** — journal entries, ledger balances, and audit logs are immutable once committed.
- AI explains but never becomes the system of record** — all AI output is labeled as generated; AI never posts journals or mutates financial state.
- Audit trails are unbreakable** — every mutation calls `recordAudit()` or `recordIAMAudit()` with SHA-256 hash chaining.

2. System Architecture

2.1 Seven-Layer Architecture

Perionyx is organized into seven logical layers:

Edge / Security Layer
Proxy (src/proxy.ts), Rate Limiting, CSRF, Auth
Application Layer
React Server Components, Client Components, App
API Layer
272+ REST Endpoints, zod Validation, Cache-Control
Business Logic Layer
56 Business Modules, Domain Services, Facades
Integration Layer
Connector Framework, Plaid, ACH, Webhooks
Intelligence Layer
6 Deterministic Engines, KPIs, Trends, Scorecards
Infrastructure Layer
CacheManager, QueueManager, LockManager, Metrics

2.2 Proxy Architecture

Next.js 16 uses `src/proxy.ts` instead of `src/middleware.ts`. The proxy handles:

- Request ID generation (UUID correlation IDs)
- Locale detection (`NEXT_LOCALE` cookie → `Accept-Language` header)
- Rate limiting (Redis-backed, in-memory fallback)
- Auth extraction (JWT from cookies, API key from Bearer header)
- CSRF validation on mutation requests

Source: `docs/architecture/02-system-architecture.md`

2.3 Key Architectural Decisions

Decision	Rationale
Proxy replaces Middleware	Next.js 16 uses <code>src/proxy.ts</code> at edge
Unified error format	All 272+ endpoints use <code>handleRouteError()</code> / <code>zodErrorResponse()</code>
Cache headers on 18 endpoints	Tiered TTLs (15-120s) for CDN adoption
Stale-while-revalidate	Background refresh of cached financial data
In-memory stores (ephemeral)	Rules/schedules/matrix stored in <code>Map</code> objects; DB persistence planned

3. Technology Stack

3.1 Core Framework

Layer	Technology	Version
Framework	Next.js	16.2.6
UI Library	React	19.2.4
Language	TypeScript	5.9.3 (strict mode)
ORM	Prisma	7.8
Database	PostgreSQL	16
Queue	PgBoss	12.24
Auth	NextAuth (Auth.js)	5.0.0-beta.31
Styling	Tailwind CSS	4.3
Animation	framer-motion	12.42.1
Validation	zod	4.4.3
Logging	Pino	10.3.1
Package Manager	pnpm	latest

3.2 Infrastructure Components

Component	Implementation	Purpose
CacheManager	LRU in-memory + Redis	Tiered caching with namespaced keys
LockManager	In-memory + Redis	Distributed locks with lease renewal
QueueManager	Facade over PgBoss	8 default queues, dead-letter routing
MetricsRegistry	Counter/Gauge/Histogram	Prometheus exposition
HealthRegistry	5 standard checks	Cache, memory, uptime, queues, persistence

3.3 Why These Technologies

Decision	Rationale
PostgreSQL over MongoDB	Transactional integrity required for double-entry accounting
PgBoss over Redis-only queues	PostgreSQL durability without separate queue infrastructure
REST over GraphQL	Simpler for financial data queries; Cache-Control headers suffice
Tailwind over CSS Modules	Utility approach reduces bundle size, enforces design consistency
framer-motion	AnimatePresence for mount/unmount, gesture support for workflow designer

Source: <docs/architecture/03-technology-stack.md>

4. Domain Architecture & Business Modules

4.1 Domain Map

The platform's 56 business modules are organized into five domains:

Identity & Security
IAM, RBAC, ABAC, MFA, SSO, Security Headers, Encryption
Financial Core
Treasury, Ledger, Reporting, Risk
Intelligence
6 Engines: Anomaly, Forecast, Recommendations, Trends, Compliance, NLP Commentary
Operations
Integrations, Workflow Orchestration, Queue, Notifications
Experience
Enterprise Experience, Automation Studio, Onboarding

4.2 Module Interaction Pattern

All modules communicate through typed function calls. Cross-domain operations use the service facade pattern. Circular dependencies are prohibited. Each module exports a barrel (`index.ts`) and typically contains `*.service.ts` , `*.types.ts` , and supporting files.

Domain	Location	Key Files
Automation Studio	<code>src/modules/automation-studio/</code>	11+ files
Ledger	<code>src/modules/ledger/</code>	10 files
Treasury	<code>src/modules/treasury/</code>	3 files
Intelligence Platform	<code>src/modules/intelligence-platform/</code>	14 files
Workflow Orchestration	<code>src/modules/orchestration/</code>	13 files
Enterprise Experience	<code>src/modules/enterprise-experience/</code>	10 files
Notifications	<code>src/modules/notifications/</code>	6 files
Queue System	<code>src/modules/queue/</code>	13 files

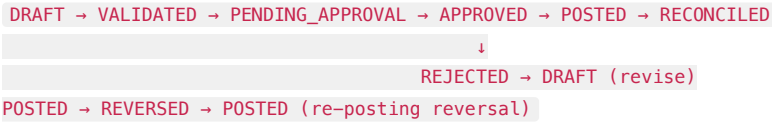
Source: `docs/architecture/04-domain-architecture.md`

5. Financial Engine — Ledger & Accounting

5.1 Double-Entry Accounting

All financial transactions create balanced debit/credit pairs in `LedgerEntry`. The ledger is append-only — entries are never mutated after posting. Corrections require reversal transactions.

5.2 Journal Lifecycle



5.3 Six-Stage Processing Pipeline

Stage	Module	Responsibility
1. Validation	<code>transaction-validator.ts</code>	Balance integrity, account validity, tenant scope, currency consistency, idempotency
2. State Machine	<code>transaction-state-machine.ts</code>	Enforces valid state transitions
3. Posting Engine	<code>posting-engine.ts</code>	Commits journals, updates balances atomically
4. Approval Workflow	<code>approval-workflow.ts</code>	Routes to approvers, sequential/parallel paths, escalation
5. Reconciliation	<code>reconciliation-engine.ts</code>	Matches entries to external statements
6. Reversal	<code>reversal-engine.ts</code>	Creates offsetting entries (never mutates originals)

5.4 Invariant

`Sum(debits) - Sum(credits) = 0` (within currency precision tolerance)

Enforced at validation, posting, and reconciliation. Violation causes full batch rollback.

Source: `docs/architecture/05-financial-engine.md`

6. Treasury Platform

6.1 Core Models

Model	Purpose
TreasuryCashPosition	Current cash holdings by account and currency
TreasuryLiquidityPosition	Liquidity tiers (operational, reserve, surplus)
TreasuryCashPool	Grouped cash accounts for consolidated visibility
TreasuryCashMovement	Individual cash inflows and outflows
TreasuryCashForecast	Projected cash positions over time horizons
TreasuryFXExposure	Foreign currency exposure by currency pair
TreasuryCounterpartyRisk	Counterparty credit risk assessments
TreasuryWorkingCapital	DSO, DPO, CCC metrics

6.2 Cash Management

- Position aggregation by currency, account type, entity
- Pool visibility for consolidated viewing
- Movement tracking with audit trail
- Restricted cash tagging (regulatory, collateral)

6.3 Liquidity Tiers

Tier	Description	Examples
Operational	Daily operations	Payroll, vendor payments
Reserve	Contingencies	Regulatory buffer, emergency fund
Surplus	Available for investment	Short-term instruments, intercompany loans

6.4 Bank Integrations

Two channels:

1. **External Banking Service** (`external-banking.service.ts`) — abstracted bank protocol for balance/transaction/wire/statement
2. **Plaid Integration** — account verification, transaction categorization, real-time balances

6.5 Cross-Domain Interactions

Domain	Interaction
Ledger	Treasury cash movements post as journal entries
Reporting	Cash position data feeds board packs
Intelligence Platform	Cash flow patterns feed AI forecasting
Integrations	Bank data flows through connector framework

Source: <docs/architecture/06-treasury-platform.md>

7. Reporting & Analytics Platform

7.1 Report Engine Pipeline

- 1. **Data retrieval** — Ledger balances, treasury positions, time-series data
- 2. **Computation** — Variance, ratios, consolidations
- 3. **Formatting** — Target output structure
- 4. **Presentation** — Analytics components or exports

7.2 Supported Reports

Report	Source	Frequency
Profit & Loss	Revenue/expense accounts	Monthly, quarterly
Balance Sheet	Asset/liability/equity accounts	Monthly, quarterly
Cash Flow Statement	Ledger + Treasury cash movements	Monthly, weekly
Variance Report	Budget vs actual comparison	Monthly
Board Pack	Combined + AI commentary	Quarterly

7.3 13 Analytics Components

All rendered as custom inline SVG — no external chart libraries:

Component	Purpose
<code>ExecutiveKpiCard</code>	Metric display with trend, sparkline, color-coded status
<code>CashFlowTimeline</code>	Time-series with forecast boundary
<code>VarianceCard</code>	Budget vs actual variance
<code>ForecastChart</code>	Projection with confidence intervals
<code>ApprovalAnalytics</code>	Volume, cycle time, approval rate
<code>WorkflowAnalytics</code>	Step durations, bottlenecks, failure rates
<code>DrillDownPanel</code>	Hierarchical data exploration
<code>InsightPanel</code>	AI-generated insights

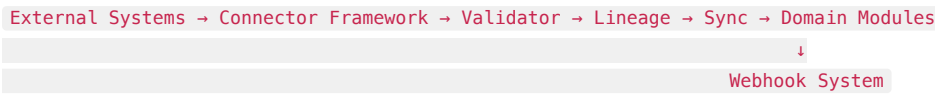
7.4 Rendering Strategy

- Metric values render first (synchronously from cache)
- Charts render second (asynchronously)
- Cache tiering: critical (5s), standard (60s), historical (600s)
- CSV (UTF-8 BOM) and XML Excel exports

Source: `docs/architecture/07-reporting-platform.md`

8. Integration Platform

8.1 Architecture



8.2 Connector Framework

All connectors implement `IConnector` interface:

- `connect()`, `disconnect()`, `healthCheck()`, `sync()`, `validate()`

8.3 Built-in Connectors

Connector	Type	Purpose
Plaid	Bank connectivity	Account verification, balances, transactions
ACH	Payment processing	Automated Clearing House payments
HTTP	Generic REST	Configurable authentication
Mock	Testing	Simulated sandbox data

8.4 Data Pipeline

Every data point passes through: Schema validation → Business validation → Duplicate detection → Sanity checks → Canonical transformation → Lineage recording → Domain storage → Webhook notification

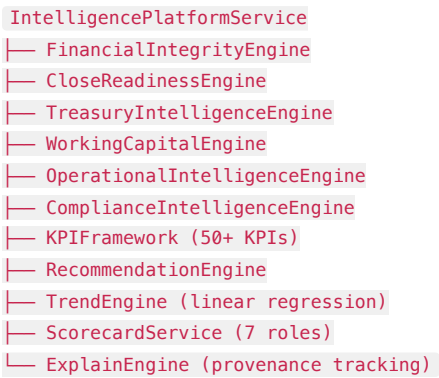
8.5 Lineage & Audit

Every data point tracked by: source system, original ID, sync timestamp, transformation log, connector version. Enables full traceability — every number traces back to its external source.

Source: `docs/architecture/08-integration-platform.md`

9. Intelligence Platform

9.1 Architecture



9.2 Six Deterministic Engines

Engine	Key Metrics	Weighted Components
Financial Integrity	GL balances, journals, reconciliations	6 components (25% ledger, 25% reconciliation, ...)
Close Readiness	Closing checklists, unreconciled items	Period-end readiness scoring
Treasury Intelligence	Cash position, liquidity, FX, risk	Multi-factor treasury health
Working Capital	AR/AP aging, DSO, DPO, CCC	Working capital efficiency
Operational Intelligence	Connector health, sync latency, errors	Platform operational health
Compliance Intelligence	Policy violations, audit gaps, frameworks	Regulatory compliance posture

9.3 Engine Pattern

Every engine follows: `static async calculate(ctx: TenantContext): Promise<EngineResult>`

- Returns 0-100 composite score, severity, evidence map, recommendations
- All calculations are deterministic — same inputs always produce same outputs
- Zero LLM/AI involvement in factual financial scoring

9.4 KPI Framework

50+ KPIs across 5 categories. Each KPI has: key, label, unit, category, current/previous/target values, threshold-based status (`on_track` , `at_risk` , `critical` , `neutral`), trend direction.

9.5 Scorecard Service

Role-specific snapshots for: `ceo` , `cfo` , `controller` , `treasurer` , `finance-manager` , `board` , `auditor` . Each includes all six domain scores, role-filtered KPIs, and recommendations.

9.6 Explain Engine

Provenance tracking linking any score/KPI/recommendation back to source data (report, ledger, journal, transaction, document, integration, approval, audit). Supports bidirectional lookups.

Source: `docs/architecture/09-intelligence-platform.md`

10. Workflow Orchestration

10.1 Components

Component	Function
<code>WorkflowEngine</code>	Execute workflows with sequential step dispatch
<code>WorkflowBuilder</code>	CRUD for workflow definitions
<code>AutomationEngine</code>	IF/THEN rules evaluated on events
<code>SchedulerService</code>	Cron-based scheduling with timezone support
<code>TemplateLibrary</code>	10 built-in workflow templates
<code>MonitorService</code>	Execution monitoring, cancel, retry
<code>InternalEventBus</code>	Publish/subscribe with emit/on/off
<code>OrchestrationService</code>	Facade over all subsystems

10.2 Step Types

Type	Behavior
<code>module_action</code>	Dispatches to named module (treasury, ledger, reporting, etc.)
<code>condition</code>	Evaluates configurable condition; can halt on failure
<code>notification</code>	Sends notifications via event bus
<code>delay</code>	Waits for configurable timeout
<code>sub_workflow</code>	Executes child workflow

10.3 Workflow Templates (10)

Template	Steps
Month-End Close	6 steps: trial balance → reconciliation → condition → statements → approval → notification
Daily Treasury Review	4 steps: cash position → FX rates → liquidity → notification
Weekly Executive Brief	4 steps: KPIs → variance report → recommendations → notification
Bank Reconciliation	4 steps: sync bank → match → condition → notification
Cash Forecast Refresh	5 steps: cash → receivables → payables → forecast → notification
Budget Review	5 steps: actuals → budget → variance → report → notification
Board Pack Generation	4 steps: financials → board pack → commentary → distribution
Quarter-End Close	7 steps
Year-End Close	8 steps
Audit Preparation	6 steps

10.4 Trigger Types

Manual, Event-driven, Schedule (cron), and Automation (IF/THEN rules).

Source: `docs/architecture/10-workflow-orchestration.md`

11. Enterprise Experience Platform

11.1 Role Dashboards

Role-specific configurations for 9 roles:

Role	Layout	Focus
CFO	3 columns	Cash position, approvals, forecast, risk
Controller	3 columns	Reconciliation, month-end, exceptions
Treasurer	3 columns	Cash, liquidity, FX, forecast
Finance Manager	2 columns	Team activity, approvals, calendar
AP/AR	Domain-specific	Invoice queue, aging, disputes
Auditor	Audit-focused	Trail, compliance, access review
Executive	Summary	Board pack, strategic KPIs
Administrator	System	Health, users, integrations

11.2 Enterprise Form System (14 components)

- `EnterpriseForm` — root wrapper with auto-save (debounced 2s)
- `EnterpriseSection` — collapsible with error badge, advanced badge
- `EnterpriseField` — standardized label + input + error + help + hint
- `SmartSelect` — searchable, grouped multi-select
- `ConditionEditor` — field/operator/value builder
- `ApprovalPreview` — visual approval path simulation
- `EnterpriseWizard` — multi-step with step indicator
- `ValidationSummary` , `AutoSaveIndicator` , `UnsavedChangesGuard`

11.3 Enterprise Motion (13 components)

Standardized animation tokens: durations 100-600ms, 6 easing curves, 12 variants.

- `AnimatedCard` , `AnimatedButton` , `AnimatedDialog` , `AnimatedToast`
- `AnimatedMetric` (counter animation), `AnimatedSidebar`
- `AnimatedTable` + `AnimatedTableRow` (stagger)
- `PageTransition` , `SectionTransition`
- `LoadingSkeleton` with shimmer variants

11.4 Mobile Experience (9 components)

- `MobileMetricCard` , `ApprovalQuickView` , `MobileNotificationCenter`
- `QuickActionBar` , `AdaptiveNavigation` , `TouchToolbar`
- `OfflineIndicator` , `ConnectionStatus`
- Pages: `/mobile-dashboard` , `/mobile/treasury`
- Bottom navigation: Overview/Approvals/Treasury/Alerts/Insights

11.5 Onboarding Module

10-step wizard with 12-domain readiness verification:

1. Company Setup → 2. Organization Structure → 3. User Invitation → 4. Treasury Setup → 5. Workflow Configuration → 6. Integration Setup → 7. Governance → 8. AI Configuration → 9. Intelligence → 10. Enterprise Readiness (scored report with pass/warn/fail badges)

Source: `docs/architecture/11-experience-platform.md`

12. Security Architecture

12.1 Authentication

Mechanism	Implementation
Framework	NextAuth v5 with Prisma adapter
Sessions	JWT, 30-day <code>maxAge</code> , <code>httpOnly</code> / <code>secure</code> cookies
API Keys	<code>va_{0-9a-f}{64}</code> format, validated via <code>ApiKeyService</code>
Account Lockout	<code>failedLoginAttempts</code> + <code>lockedUntil</code> on User model
Password Hashing	bcrypt via Prisma <code>password_hash</code> field

12.2 Authorization — RBAC + ABAC

- **RBAC:** 46+ `GranularPermission` types, 17 enterprise role definitions (system_administrator through read_only_executive)
- **ABAC:** Attribute-based policies with conditions (`eq`, `neq`, `gt`, `gte`, `lt`, `lte`, `in`, `contains`)
- **Sandbox Restrictions:** 10 permissions blocked in sandbox mode
- **Enforcement:** `requirePermission()` at every API handler

12.3 Endpoint Protection Pattern

```
requirePermission(request, "permission.name")
└─ auth() → verify session
└─ API key fallback for Bearer tokens
└─ requireTenantContext() → tenant isolation
└─ rbacService.ensurePermission() → permission check
└─ Returns AuthContext { userId, companyId, role }
```

12.4 Encryption

AES-256-GCM with key rotation support:

- 256-bit key, 16-byte IV, 16-byte auth tag
- Key rotation via `ENCRYPTION_KEY_HISTORY` env var
- Payload format: `base64(metadata):hex(ciphertext)`
- KMS provider interface for cloud HSM integration

12.5 Audit Logging

- Tamper-evident via SHA-256 hash chaining
- Every entry includes `previousHash` linking to prior entry
- Severity levels: `info`, `warning`, `critical`
- Standard fields: `userId`, `companyId`, `action`, `resource type/ID`, `IP`, `correlationId`

12.6 Security Headers

CSP (strict self-origin), HSTS (env-based max-age), X-Content-Type-Options (nosniff), X-Frame-Options (DENY), Referrer-Policy, Permissions-Policy (camera/microphone/geolocation disabled), COOP/COEP/CORP (same-origin).

Source: `docs/architecture/12-security-architecture.md`

13. Multi-Tenancy Architecture

13.1 Tenant Model

The `Company` model is the fundamental tenant boundary. Every data table carries `companyId` with foreign key to `Company`. All queries are filtered by `companyId` at the application layer.

13.2 Tenant Context Propagation

```
type TenantContext = { userId: string; companyId: string; role: CompanyRole };
```

The `requireTenantContext()` guard enforces:

1. Authentication (throws `UnauthorizedError`)
2. Active company (throws `ForbiddenError`)
3. License enforcement (optional `LICENSE_COMPANY_ID` env var)

13.3 Cross-Tenant Blocking (Four Layers)

1. **Service-level:** every query includes `where: { companyId: ctx.companyId }`
2. **Update/Delete guards:** resources fetched first, checked for `companyId` match
3. **API route enforcement:** `requireTenantContext()` before handler logic
4. **Workflow execution:** engine checks `def.companyId !== ctx.companyId`

13.4 Scalability

The `companyId` on every table supports three deployment models without schema changes: shared table (current), schema-per-tenant (future), database-per-tenant (future).

Source: `docs/architecture/13-multi-tenancy.md`

14. Database Architecture

14.1 Schema Overview

File: `prisma/schema.prisma` (~4,548 lines, 100+ models)

Domain Area	Key Models
Auth	User, Account, Session, VerificationToken, Authenticator
Tenancy	Company, CompanyMembership, OrganizationUnit
Treasury	15 models (CashPosition, Liquidity, Forecast, FX, Risk, etc.)
General Ledger	20+ models (GLAccount, GLJournal, GLJournalEntry, GLPostingBatch, GLCostCenter, etc.)
Ledger	LedgerEntry, Transaction, Wallet, ExternalAccount
Reporting	FinancialReportDefinition, BoardPack, SavedViews
Intelligence	FinancialScore, KPIValue, IntelligenceRecommendation, Trend, Scorecard
Orchestration	WorkflowDefinition, Execution, StepExecution, Template, AutomationRule
Governance	Policy, PolicyViolation, ApprovalRule, ApprovalThread
Audit	AuditLog, IdempotencyRecord

14.2 Schema Design Principles

- `companyId` on every data table for universal tenant isolation
- Cascade deletes for clean tenant teardown
- Separate model groups per domain
- Financial rules in services, not schema
- No cross-model constraints — relational integrity at service layer

14.3 Index Strategy

- `companyId` indexed on every tenant-scoped table
- Composite indexes for common query patterns
- Unique constraints for natural keys
- Foreign key indexes on all relations

14.4 JSONB Usage

Flexible fields in: workflow steps, score components, automation rule conditions, KPI metadata, insight event context, company metadata.

Source: `docs/architecture/14-database-architecture.md`

15. Queue & Async Processing

15.1 Two-Layer Queue Architecture

1. **Production Job Queue** ([src/modules/queue/](#)) — PgBoss-based with 11 typed job types
2. **Infrastructure Queue Manager** ([src/server/queues/](#)) — 8 pre-configured queues with worker pools

15.2 Job Types (11)

Job Type	Payload Type	Purpose
notification-email	NotificationJobPayload	Email delivery
notification-slack	NotificationJobPayload	Slack delivery
notification-connector	NotificationJobPayload	Connector delivery
workflow-execute	WorkflowJobPayload	Workflow execution
ai-analysis	AiJobPayload	Financial analysis
ai-forecast	AiJobPayload	Forecasting
ai-narrative	AiJobPayload	Narrative generation
ai-extraction	AiJobPayload	Data extraction
ai-risk	AiJobPayload	Risk assessment
report-generate	ReportJobPayload	Report generation
connector-sync	ConnectorSyncJobPayload	Connector sync

15.3 Default Infrastructure Queues

Queue	Type	Concurrency	Rate Limit	Max Retries	DLQ
sync	FIFO	5	100/min	3	sync-dlq
forecast	Priority	3	20/min	2	forecast-dlq
payment	FIFO	10	200/min	5	payment-dlq
notification	FIFO	20	500/min	3	notification-dlq
alert	Priority	5	100/min	3	alert-dlq
metrics	FIFO	2	50/min	1	metrics-dlq
audit	FIFO	3	200/min	2	audit-dlq
recommendation	Priority	2	10/min	1	recommendation-dlq

15.4 Key Features

- Exponential backoff with jitter on retries
- Dead-letter routing after max retries
- Concurrency control per queue
- Cron-based and delayed scheduling
- Batch enqueue support

Source: [docs/architecture/15-queue-system.md](#)

16. API Architecture

16.1 Handler Pattern

Every API V1 handler follows:

```
export async function GET(request: Request) {
  try {
    const ctx = await requirePermission(request, "permission.name");
    const data = await fetchData(ctx.companyId, searchParams);
    return NextResponse.json(data, {
      headers: { ...cacheHeaders(60), "x-request-id": request.headers.get("x-request-id") ?? "" },
    });
  } catch (error) {
    return handleRouteError(error, request);
  }
}
```

16.2 Request Flow

Client → Edge Proxy → requirePermission → Business Logic → PostgreSQL
(Rate limit, CSRF, (Auth, Tenant, (Service layer,
Locale, Correlation) Permission) Prisma queries)

16.3 Unified Error Handling

Function	Purpose
<code>handleRouteError(error, request)</code>	Catches <code>AppError</code> or generic errors → structured JSON
<code>zodErrorResponse(error, request)</code>	Zod validation failures → 400 with issues array
<code>parseJsonBody<T>(request)</code>	JSON body parsing with error handling

Error codes: `UNAUTHORIZED` , `FORBIDDEN` , `VALIDATION` , `NOT_FOUND` , `CONFLICT` , `TOO_MANY_REQUESTS` , `INTERNAL`

16.4 Cache Strategy

Method	Headers
GET	<code>cacheHeaders(ttl)</code> — public, <code>s-maxage</code> 15-120s, <code>stale-while-revalidate</code> 10x TTL
POST/PUT/PATCH/DELETE	<code>noCacheHeaders()</code> — no-store, must-revalidate

16.5 API Endpoint Count

40+ endpoint directories covering: treasury, ledger, intelligence, orchestration, IAM, connectors, reporting, risk, enterprise experience, and more.

Source: `docs/architecture/16-api-architecture.md`

17. Deployment Architecture

17.1 Docker Multi-Stage Build

Stage	Base	Purpose
deps	node:22-alpine	<code>pnpm install --frozen-lockfile</code>
builder	node:22-alpine	<code>pnpm build</code>
runner	node:22-alpine	Next.js standalone server (non-root user, port 3000)

17.2 Health Endpoints

Endpoint	Function
<code>/health</code>	Overall status with 5 checks (cache, memory, uptime, queues, persistence)
<code>/readiness</code>	Binary ready/not-ready
<code>/liveness</code>	Process alive check

17.3 Graceful Shutdown

SIGTERM/SIGINT interception → connection draining → in-flight request completion (30s timeout) → queue worker stop → cache flush.

17.4 Recovery Module

Module	Function
Backup Manager	Scheduled and on-demand database backups
Restore Manager	Point-in-time and full restore
Snapshot Manager	Application state snapshots
Recovery Validator	Post-restore integrity validation

17.5 CI/CD Pipeline

CI (`ci.yml`): TypeScript check → Lint → Test → Build → Security scan → Migration verify **CD** (`deploy.yml`): Docker build/push → Prisma migrate → k8s apply → Health check → Automated rollback

17.6 Monitoring

- Prometheus metrics at `/api/v1/enterprise/metrics` (8 domains)
- Grafana dashboards for app health, queues, cache, database, business metrics
- Structured JSON logging via Pino with correlation IDs

Source: `docs/architecture/17-deployment-architecture.md`

18. Scalability & Performance Strategy

18.1 Caching Architecture

Tier	Provider	Latency	Use Case
L1	LRU in-memory (<code>Map</code>)	<1ms	Sessions, permissions, feature flags
L2	Redis	<5ms	Metrics, dashboards, cached queries

18.2 TTL Strategy

Category	TTL	Stale TTL	Examples
Critical	5s	600s	Wallet balances, cash positions
Fast	30s	300s	Transaction lists, approval queues
Standard	120s	600s	Audit logs, report data
Stale	600s	1800s	Historical metrics, static config

18.3 Database Optimization

- 18 performance indexes across high-query tables
- Cursor-based pagination (default 25, max 200)
- JSONB for flexible fields
- Parallel `Promise.all` for independent queries

18.4 Lazy Loading Strategy

- React Server Components for data-fetching pages
- Dynamic imports for charting, tables, forms
- Route-based code splitting via App Router

18.5 Future Scalability

- Postgres read replicas for GET endpoints (planned)
- Schema-based sharding by tenant group (planned)
- Distributed Redis-backed rate limiting (planned)

Source: `docs/architecture/18-scalability-performance.md`

19. AI Governance & Intelligence

19.1 Non-Negotiable Principle

Financial facts are always produced by deterministic systems. AI may explain, summarize, recommend, or investigate — but it must never become the system of financial record.

19.2 What AI May Do

Capability	Governance
Explain financial data	Must cite data sources
Summarize trends and reports	Must indicate data freshness
Recommend actions based on data	Must include evidence and confidence
Investigate transaction flows	Must show audit trail references
Generate commentary	Must distinguish AI text from deterministic output

19.3 What AI May Never Do

Activity	Why
Produce journal entries	AI cannot create financial records
Approve transactions	Only humans authorize financial actions
Modify financial records	Immutability of financial history
Generate financial statements as records of truth	Statements must be deterministic from ledger data
Make predictions without confidence labels	Financial decisions need uncertainty communication

19.4 Evidence Requirements

Every AI recommendation must include: summary, details, confidence level (**high** / **medium** / **low**), and evidence array with source module, record ID, field, label, timestamp, and score type.

19.5 Confidence Levels

Level	Source	UI Treatment
high	Live database query	Green indicator
medium	Cached query or historical snapshot	Amber indicator
low	AI training data (general knowledge)	Red indicator with warning
simulated	Sandbox/demo environment	Gold indicator

19.6 Enterprise Trust Model

System of Record (Deterministic) → AI Layer (Explanatory) → Human Layer (Authority)

PostgreSQL	Insights	CFO
Double-Entry Ledger	Commentary	Controller
Audit Trail	Recommendations	Auditor
Risk Engine	Investigation	Finance Manager

Source: [docs/architecture/19-ai-governance.md](#) , [docs/AI_GUIDELINES.md](#)

20. Engineering Standards & Constitutions

20.1 Perionyx Engineering Constitution

The foundational governance document ([docs/architecture/perionyx-engineering-constitution.md](#)) establishes:

Core Philosophy:

- **Correctness Over Performance** — fast but wrong is worthless
- **Immutable by Default** — financial records never mutated
- **Fail Closed** — when uncertain, don't proceed
- **Explicit Over Implicit** — every transaction boundary explicit
- **Defense in Depth** — pessimistic + optimistic locking on every balance write

Financial Integrity Principles:

- Double-entry accounting enforced at application layer
- Immutable ledger (`LedgerEntry` never updated/deleted)
- Version-protected balances (`updateMany` with version condition)
- Non-negative enforcement for STANDARD wallets
- Hybrid locking (FOR UPDATE pessimistic + version-check optimistic)

Transaction Principles:

- Explicit `Prisma.$transaction` for financial writes
- Row locking before read
- Retry with exponential backoff (3 retries, 100ms base, 3s max)
- 30-second timeout on financial transactions
- I/O operations moved outside transaction boundaries

Coding Standards:

- TypeScript strict mode, zero errors at build
- No `any` types
- No raw SQL in financial logic
- Barrel exports per module
- Typed errors (`ConflictError` , `ValidationError` , `NotFoundError` , `ForbiddenError`)

Definition of Done (12 criteria): Zero TypeScript errors, tests pass, concurrency verified, audit trail verified, tenant isolation verified, idempotency verified, documentation updated, migration forward-only, production build succeeds, code reviewed, Performance Impact Assessment completed, Enterprise Value Assessment completed.

20.2 Performance Constitution

Supplementary document ([docs/architecture/performance-constitution.md](#)) covering:

- **Frontend:** Prefer Server Components, minimize client components, lazy load heavy modules, virtualize large tables, debounce search, use loading skeletons
- **Backend:** Never perform N+1 queries, batch operations, select only required columns, always paginate, cache expensive reads, run long work async
- **Database:** Review every new table for indexes, keep transactions short, use row locking only when required, archive historical data
- **Workflow:** Never execute expensive workflows synchronously, queue long-running jobs, stream progress
- **Connectors:** Sync incrementally, retry intelligently, avoid full re-syncs
- **AI:** Cache repeated prompts, stream responses, support cancellation, track latency

20.3 AI Engineering Playbook

[docs/architecture/ai-engineering-playbook.md](#) governs all AI-assisted development:

- **Service Layer Invariance:** Always reuse existing services, never duplicate business logic
- **PostingEngine Monopoly:** All wallet posting through `PostingEngine.postLedgerLines()`
- **Lock Manager Discipline:** Every financial write uses `RowLockManager`
- **FinancialTransactionManager Invariance:** All retry-wrapped transactions through `FinancialTransactionManager`

- **Audit Trail Mandate:** Every financial action produces an `AuditLog` record
- **Version Check Integrity:** All balance updates use `updateMany` with version condition + atomic increment

20.4 Self-Review Framework

`docs/architecture/self-review-framework.md` — mandatory pre-completion checklist covering:

1. Architecture (8 criteria)
2. Financial Integrity (9 criteria)
3. Transaction Safety (8 criteria)
4. Concurrency (7 criteria)
5. Tenant Isolation (8 criteria)
6. Security & Audit (11 criteria)
7. Error Handling (5 criteria)
8. Testing (7 criteria)

20.5 Enterprise Readiness Checklist

`docs/architecture/enterprise-readiness-checklist.md` — quality gate with 366 criteria across 12 domains: Architecture, Multi-Tenancy, Financial Integrity, Concurrency, Transaction Safety, Security, Audit, Error Handling, Testing, API, Documentation, Monitoring.

21. Platform Roadmap

21.1 Completed Phases

Phase	Description	Status
7D	Onboarding Module & Enterprise Readiness	Complete
7E	Enterprise Persistence Infrastructure	Complete
7F	Enterprise Production Readiness Platform	Complete
8A	Enterprise Performance & Optimization	Complete
8B.4	Enterprise Tables 2.0	Complete
8B.5	Executive Data Visualization & Analytics	Complete
8B.6	Enterprise Forms & Workflow UX	Complete
8B.7	Enterprise Motion & Micro-Interactions	Complete
8B.8	Executive Mobile Experience	Complete
8B.9	Enterprise Accessibility & UX Polish	Complete
11B	Enterprise Installation & Deployment Platform	Complete
11C	Enterprise Identity & Access Management	Complete
12A	Financial Reporting & Integration Platform	Complete
12B	Enterprise Experience	Complete
12C	Financial Intelligence	Complete
12D	Platform Orchestration	Complete
12E	Architecture Documentation	Complete

21.2 Future Phases

Phase	Description	Target
8C	Distributed Rate Limiting & Read Replicas	2026-Q4
9C	Investments (portfolio, buckets, restricted cash)	2027-Q1
9D	Risk (advanced scoring, stress testing, scenario analysis)	2027-Q1
9E	Compliance (automated monitoring, regulatory reporting)	2027-Q2
9F	Executive AI (multi-turn conversation, proactive alerts, ML)	2027-Q2

Source: [docs/architecture/21-roadmap.md](#) , [docs/releases/v1.0-platform-core.md](#)

22. Documentation Index & Cross-References

22.1 Architecture Documents

Document	Code	Summary
Executive Overview	01-executive-overview.md	Platform vision, philosophy, key metrics
System Architecture	02-system-architecture.md	Seven layers, multi-tenancy, domain boundaries
Technology Stack	03-technology-stack.md	Everything selected and why
Domain Architecture	04-domain-architecture.md	56 modules, interactions, sequence diagrams
Financial Engine	05-financial-engine.md	Double-entry, journal lifecycle, 6-stage pipeline
Treasury Platform	06-treasury-platform.md	Cash, liquidity, FX, risk, forecasting
Reporting Platform	07-reporting-platform.md	Financial statements, analytics, exports
Integration Platform	08-integration-platform.md	Connectors, Plaid, webhooks, data lineage
Intelligence Platform	09-intelligence-platform.md	6 engines, KPIs, trends, scorecards
Workflow Orchestration	10-workflow-orchestration.md	Engine, templates, automation, scheduler
Experience Platform	11-experience-platform.md	Roles, forms, motion, mobile, onboarding
Security Architecture	12-security-architecture.md	Auth, RBAC, ABAC, encryption, audit
Multi-Tenancy	13-multi-tenancy.md	Tenant model, isolation, context propagation
Database Architecture	14-database-architecture.md	Schema, indexes, JSONB, migrations
Queue System	15-queue-system.md	PgBoss, job types, infrastructure queues
API Architecture	16-api-architecture.md	Handler pattern, error format, caching
Deployment Architecture	17-deployment-architecture.md	Docker, k8s, CI/CD, health, recovery
Scalability & Performance	18-scalability-performance.md	Cache tiers, TTL strategy, optimization
AI Governance	19-ai-governance.md	Principles, trust model, evidence requirements
Engineering Standards	20-engineering-standards.md	Folder structure, naming, testing, errors
Platform Roadmap	21-roadmap.md	Completed phases, future timeline
Documentation Index	22-documentation-index.md	Central navigation for all docs
Architecture Diagrams	23-architecture-diagrams.md	8 Mermaid diagrams
Architecture Freeze v1	architecture-freeze-v1.md	15 frozen components, change management

22.2 Supporting Documents

Document	Location	Purpose
Engineering Constitution	docs/architecture/perionyx-engineering-constitution.md	Immutable engineering principles
Performance Constitution	docs/architecture/performance-constitution.md	Performance principles across all layers
AI Engineering Playbook	docs/architecture/ai-engineering-playbook.md	AI-assisted development rules
Enterprise Readiness Checklist	docs/architecture/enterprise-readiness-checklist.md	366-criteria quality gate
Self-Review Framework	docs/architecture/self-review-framework.md	Pre-completion mandatory review
Transaction Strategy	docs/architecture/transaction-strategy.md	ACID guarantees, isolation levels, locking patterns
Product Constitution	docs/PRODUCT_CONSTITUTION.md	Product principles and decision framework
AI Guidelines	docs/AI_GUIDELINES.md	AI product guidelines, grounding, response structure
API Guidelines	docs/API_GUIDELINES.md	REST conventions, error format, auth
AGENTS.md	AGENTS.md (root)	Engineering constitution for AI assistants

22.3 Design Documents

Document	Location
Design System	docs/design/design-system.md
Enterprise Forms	docs/design/enterprise-forms.md
Enterprise Table System	docs/design/enterprise-table-system.md
Motion System	docs/design/motion-system.md
Executive Mobile Experience	docs/design/executive-mobile-experience.md
UX Accessibility Audit	docs/design/ux-accessibility-audit.md
Component Library	docs/design/component-library.md

22.4 Domain Documentation

The platform includes comprehensive documentation for every major domain:

- [docs/accounting/](#) — 13 documents (architecture, ledger, periods, close, consolidation, etc.)
- [docs/treasury/](#) — Treasury-specific docs
- [docs/reporting/](#) — Reporting docs
- [docs/intelligence/](#) — Intelligence docs (6+ documents)
- [docs/identity/](#) — IAM docs (12 documents)
- [docs/security/](#) — Security docs
- [docs/deployment/](#) — Deployment docs (11 documents)
- [docs/operations/](#) — Operations runbooks (8 documents)
- [docs/recovery/](#) — Recovery docs
- [docs/persistence/](#) — Persistence docs (14 documents)
- [docs/infrastructure/](#) — Infrastructure docs (8 documents)
- [docs/compliance/](#) — Compliance readiness (SOC 2, ISO 27001, PCI DSS, GDPR)
- [docs/testing/](#) — Testing strategy and coverage

23. Architecture Decision Records (ADRs)

All 30 ADRs are in `docs/adr/` :

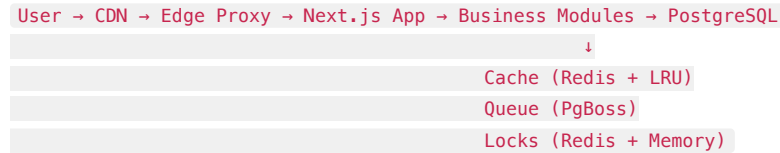
ADR	Title	Status
001	Platform Vision & Scope	Ratified
002	Multi-Tenant Architecture	Ratified
003	Authentication & RBAC	Ratified
004	Double-Entry Ledger	Ratified
005	Policy Engine	Ratified
006	Approval Engine	Ratified
007	Risk Engine	Ratified
008	Treasury Architecture	Ratified
009	Audit Architecture	Ratified
010	AI Copilot	Ratified
011	Sandbox Architecture	Ratified
012	Simulation Engine	Ratified
013	Enterprise Time Machine	Ratified
014	Integration Framework	Ratified
015	Reporting Engine	Ratified
016	Command Center	Ratified
017	Design System	Ratified
018	Security Principles	Ratified
019	Performance Strategy	Ratified
020	Future Expansion	Ratified
021	PostgreSQL	Ratified
022	Prisma ORM	Ratified
023	Next.js 16	Ratified
024	Multi-Tenancy (Application-Level)	Ratified
025	Domain Architecture (56 Modules)	Ratified
026	Integration Framework (Plugin)	Ratified
027	Workflow Orchestration	Ratified
028	Intelligence Platform (6 Engines)	Ratified
029	AI Governance	Ratified
030	Event-Driven Architecture	Ratified

Source: `docs/architecture/22-documentation-index.md` , `docs/adr/`

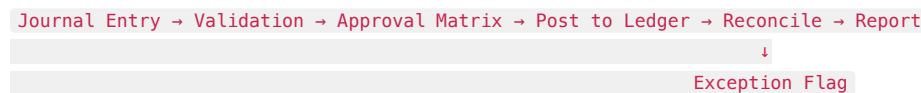
24. Architecture Diagrams

24.1 Seven-Layer Platform Architecture

All 56 business modules grouped by domain, with infrastructure and data layers:



24.2 Financial Transaction Flow



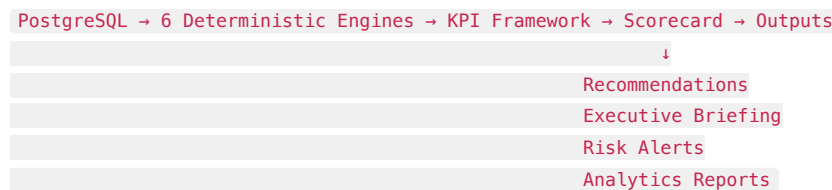
24.3 Integration Data Flow



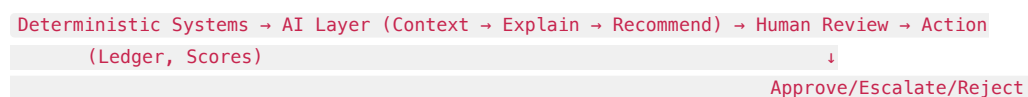
24.4 Workflow Engine



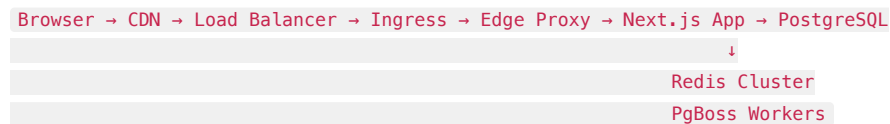
24.5 Intelligence Platform



24.6 AI Decision Flow



24.7 Deployment Topology



Full diagrams with Mermaid source: <docs/architecture/23-architecture-diagrams.md>

Appendix A: Key File Locations

Component	Location
Edge Proxy	<code>src/proxy.ts</code>
Route Handlers	<code>src/server/http/handle-route.ts</code>
Permission System	<code>src/server/iam/</code> , <code>src/modules/rbac/</code>
Encryption	<code>src/server/security/encryption.ts</code>
Cache Manager	<code>src/server/cache/</code>
Lock Manager	<code>src/server/locks/</code>
Queue Manager	<code>src/server/queues/</code>
Queue Jobs	<code>src/modules/queue/jobs/</code>
Audit Logger	<code>src/server/security/audit-logger.ts</code>
Session Manager	<code>src/server/iam/session.ts</code>
Health Endpoints	<code>src/server/ha/</code>
Backup/Recovery	<code>src/server/recovery/</code>
Persistence	<code>src/server/persistence/</code>
Infrastructure Config	<code>src/server/persistence/config.ts</code>
Infrastructure Facade	<code>src/server/infrastructure.ts</code>
Migrations	<code>prisma/migrations/</code>
Schema	<code>prisma/schema.prisma</code>
Docker	<code>Dockerfile</code> , <code>docker-compose.yml</code>
Kubernetes	<code>k8s/</code>
CI/CD	<code>.github/workflows/ci.yml</code> , <code>.github/workflows/deploy.yml</code>
CLI Installer	<code>src/cli/</code>
Setup Wizard	<code>src/app/(shell)/automation-studio/setup/</code>
Deployment Dashboard	<code>src/app/(shell)/system/deployment/</code>
Identity Pages	<code>src/app/(shell)/system/identity/</code>

Appendix B: Version & Architecture Freeze

Component	Value
Platform Version	v1.0.0
Architecture Version	v1.0
Release Name	Perionyx Platform Core
Release Date	2026-07-09
Architecture Freeze	15 components frozen — see docs/architecture/architecture-freeze-v1.md

Frozen components: Repository Pattern, Unit of Work, Transaction Management, Persistence Layer, Prisma Schema, Provider Abstraction, Treasury Architecture, Banking Architecture, Design System, Folder Structure, Coding Standards, Naming Conventions, Accessibility Standards, Documentation Standards, Domain Boundaries.

Changes to frozen components require Architecture Review Board approval for Major Version Changes (v2.0+). Minor Version Changes (v1.x) are permitted within the frozen architecture.

This handbook is the master reference for the Perionyx platform architecture. Every section cross-references its source document for detailed reading. All documents are in [docs/](#) unless otherwise noted.